

Analisi in Tempo Reale di Dati sui Voli con Apache Flink

Sistemi e Architetture per Big Data A.A. 2025/26

Valentina Jin
Ingegneria Informatica
Università di Roma Tor Vergata
Roma, Italia
valentina.jin@students.uniroma2.eu

Abstract—La presente relazione descrive la progettazione e l'implementazione di un sistema distribuito per l'analisi in tempo reale di dati sui voli aerei operati negli Stati Uniti. Sfruttando un dataset storico del Dipartimento dei Trasporti statunitense, il sistema elabora un flusso continuo di eventi per monitorare lo stato operativo delle principali compagnie aeree e identificare gli aeroporti maggiormente interessati da ritardi significativi. L'architettura si basa su Apache Flink per l'elaborazione stateful dello stream, con un'infrastruttura di data ingestion basata su Apache Kafka. L'intero sistema è containerizzato tramite Docker Compose per garantirne portabilità e riproducibilità. Le prestazioni vengono valutate sperimentalmente in termini di latenza, throughput e impatto del parallelismo.

Parole chiave— *Stream processing, Apache Flink, Apache Kafka, event time, Docker, InfluxDB, Grafana*

1. INTRODUZIONE

Il monitoraggio in tempo reale del traffico aereo richiede l'elaborazione continua di flussi di eventi relativi ai voli, con l'obiettivo di individuare tempestivamente condizioni operative anomale — ritardi, cancellazioni, congestioni su rotte o aeroporti specifici. A differenza dell'elaborazione batch, questo scenario richiede l'aggiornamento incrementale di statistiche su finestre temporali mentre gli eventi vengono osservati, gestendo al contempo l'ordine di arrivo e la semantica dell'event time.

Questo progetto implementa una pipeline di stream processing basata su Apache Flink [1] che risponde a due query sul flusso di eventi di volo. Il dataset di riferimento, fornito dal Dipartimento dei Trasporti degli Stati Uniti (BTS) [2], copre il periodo gennaio-aprile 2025 e include informazioni su compagnie aeree, orari, ritardi in partenza e in arrivo, stato del volo (cancellato/deviato) e cause di ritardo.

La gestione dell'ingestion è stata realizzata tramite Apache Kafka [3], per riprodurre un ambiente di produzione più realistico rispetto a un semplice file stream, e per abilitare esperimenti di scalabilità al variare del parallelismo.

2. ARCHITETTURA DEL SISTEMA

L'architettura del sistema è interamente containerizzata tramite Docker Compose [4], eseguita in ambiente locale su WSL2. La Fig. 1 mostra lo schema dei componenti e il flusso dei dati tra di essi.

I componenti principali sono:

- **Producer (Python):** legge i quattro file CSV mensili del dataset, calcola l'event time di ciascun volo a partire da data e orario di partenza schedulato, ordina l'intero dataset per event time crescente e ne effettua il replay verso Kafka rispettando le relazioni temporali originali, compresse da un fattore di accelerazione configurabile (TIME_SCALE_FACTOR). Durante i gap notturni privi di

voli, il producer inserisce messaggi di heartbeat fittizi per mantenere il watermark di Flink in avanzamento.

- **Apache Kafka** (broker singolo, coordinato da Zookeeper): riceve gli eventi pubblicati dal producer sul topic `flights` e disaccoppia temporalmente la sorgente dati dal motore di elaborazione.
- **Apache Flink:** costruisce il motore di elaborazione stream, distribuito su un JobManager e un numero variabile di TaskManager. Legge gli eventi dal topic Kafka, assegna watermark basati sull'event time, esegue le due pipeline di analisi (Q1 e Q2), e scrive su due canali di output paralleli: file CSV (append-only, sink a parallelismo 1 per garantire un unico file ordinato) e InfluxDB [5].
- **InfluxDB:** database time-series utilizzato come sink secondario per le metriche prodotte dalle query, alimentato in parallelo alla scrittura su file CSV.
- **Grafana** [6]: fornisce due dashboard, una per query, che interrogano InfluxDB per la visualizzazione in tempo reale dell'andamento delle statistiche prodotte da Flink.
- **Docker Compose:** orchestra l'intero ambiente multi-container, definendo la rete interna condivisa (sub-network) e i volumi persistenti; consente inoltre di scalare orizzontalmente il numero di TaskManager Flink per l'esperimento di scaling senza modificare il codice applicativo.

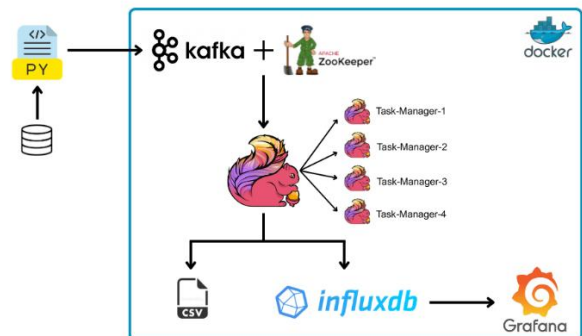


Figura 1: Schema dell'architettura del sistema

3. INGESTIONE E GENERAZIONE DEL TEMPO

A. Generazione dello Stream

Il producer Python legge i quattro file CSV mensili del dataset (202501-202504_T_ONTIME_REPORTING.csv) limitandosi ai campi strettamente necessari a Q1 e Q2 (compagnia, aeroporti, orario schedulato, ritardo, flag di cancellazione/deviazione), per contenere il volume di dati trasmesso su Kafka. Per ciascun volo l'event time viene ottenuto componendo YEAR, MONTH, DAY_OF_MONTH e CRS_DEP_TIME in un timestamp UTC in epoch millisecondi, scartando a monte le righe con CRS_DEP_TIME anomalo (fuori range 0000-2359). L'intero dataset viene quindi ordinato per event time crescente prima dell'invio, in modo che l'ordine di

trasmissione su Kafka rispecchi l'ordine cronologico dei voli indipendentemente da come questi compaiono nei file sorgente, e il replay verso Kafka rispetta le distanze temporali relative fra eventi consecutivi, comprese da un fattore `TIME_SCALE_FACTOR=86400`, equivalente a un giorno di event time per ogni secondo reale, ciò rende gli esperimenti ripetibili in tempi ragionevoli mantenendo intatte le proporzioni fra periodi a densità di traffico diversa.

I valori numerici mancanti sono trattati in due modi distinti, a seconda della semantica del campo. Un `DEP_DELAY` mancante viene propagato come `null` ed escluso dai calcoli di media (un ritardo non noto non può essere ragionevolmente approssimato a zero, che introdurrebbe un bias verso il basso); `CANCELLED` e `DIVERTED` mancanti sono invece trattati come 0, interpretando l'assenza di segnalazione esplicita come "non cancellato" o "non deviato", una scelta pensata per non perdere questi voli dal conteggio.

Il meccanismo di ingestion vero e proprio consiste nella pubblicazione, in formato JSON, su un topic Kafka a partizione singola — scelta dettata dalla necessità di preservare, lato consumer, lo stesso ordine di event time con cui il producer pubblica i messaggi: con una sola partizione l'ordine di consegna a Flink coincide con l'ordine di scrittura, lasciando come unica fonte di disordine il batching del consumer. Con partizioni multiple, la ripartizione dei messaggi su code indipendenti introdurrebbe un disordine potenzialmente più significativo tra eventi vicini nel tempo, richiedendo margini di tolleranza al watermark più ampi. Oltre ai campi del volo, ogni messaggio include un timestamp di wall-clock (`kafka_produce_time`) usato lato Flink per il calcolo della latenza end-to-end. Il consumo lato Flink avviene tramite un `KafkaSource` configurato con `OffsetsInitializer.latest()`, il che richiede che il job Flink sia già stato sottomesso e portato in stato `RUNNING` prima dell'avvio del producer; questa condizione è garantita dallo script di orchestrazione `run_pipeline.sh`, che sequenzia esplicitamente sottomissione, attesa dello stato `RUNNING` e avvio del producer.

Durante le ore notturne il traffico aereo reale è quasi nullo, e in assenza di eventi il watermark di Flink smette di avanzare, bloccando il trigger delle finestre (in particolare la finestra globale di Q2). Per ovviare a questo, quando il gap di event time fra due voli reali consecutivi supera una soglia di 5 minuti, il producer inserisce messaggi **heartbeat** fittizi (flag `heartbeat: true`, tutti gli altri campi nulli) a intervalli di 10 minuti di event time, fino a colmare il gap. Il `WatermarkStrategy` li elabora normalmente ai fini dell'avanzamento del watermark, mentre vengono scartati esplicitamente da un filtro (`isHeartbeat()`) prima di qualunque aggregazione, in modo da non alterare le statistiche di Q1 e Q2.

B. Event Time e Watermarking

L'intero sistema opera in event time: le finestre di Q1 e Q2 si chiudono in base all'avanzamento del watermark, non del clock di sistema. Sebbene il topic sia a partizione singola, un disordine residuo di pochi millisecondi può comunque presentarsi per via del batching del consumer Kafka; per assorbitirlo si adotta una strategia `BoundedOutOfOrderness` con soglia di 5 minuti di event time, dimensionata con un margine sufficiente a rimanere valida anche qualora il numero di partizioni venisse aumentato in futuro. Il watermark emesso corrisponde quindi al massimo event time osservato meno 5

minuti: una finestra $[t, t+1h)$ si chiude solo quando arriva un evento con `event time  $\geq t+1h05m$` . L'assegnazione di un evento a una finestra dipende esclusivamente dal suo event time, non dall'istante in cui viene ricevuto: un evento che arriva dopo la chiusura della finestra a cui appartiene per event time non viene quindi spostato o incluso nella finestra successiva, ma scartato definitivamente, poiché la finestra a cui apparteneva è già stata emessa e purgata. Non è configurato alcun `allowedLateness` né un side output per il recupero dei late event, per cui la soglia di 5 minuti rappresenta l'unico margine di tolleranza al disordine offerto dal sistema.

Gli heartbeat descritti precedentemente risolvono un blocco temporaneo del watermark, dovuto all'assenza di voli reali. Un secondo scenario di blocco, distinto e permanente, si presenta quando il parallelismo del job supera il numero di partizioni Kafka: alcuni subtask della sorgente non ricevono mai una partizione e restano indefinitamente privi di eventi, quindi il loro watermark resta bloccato a -infinito. Poiché i flussi paralleli di subtask confluiscono a valle, Flink deve calcolare un solo watermark complessivo che è dato dal minimo tra tutti i subtask paralleli, quindi anche un solo subtask che non riceve mai eventi bloccherebbe l'avanzamento del watermark per l'intera pipeline. Per questo motivo alla strategia è stato aggiunto un `withIdleness(10s)`, che esclude dal calcolo del minimo i subtask inattivi da più di 10 secondi di wall-clock.

4. IMPLEMENTAZIONE DELLE QUERY

Q1 — Monitoraggio delle Compagnie Aeree

Q1 calcola, per le compagnie American Airlines (AA), Delta (DL), United (UA) e Southwest (WN), statistiche orarie su voli completati, cancellati e devianti: ritardo medio in partenza, tasso di cancellazione e tasso di partenze in ritardo.

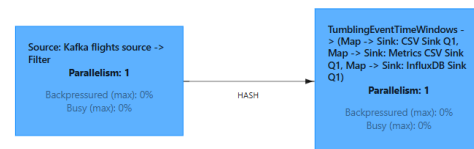


Figura 2: Schema Query 1

Il flusso di esecuzione:

1. Lettura dello stream dal topic Kafka `flights` tramite `KafkaSource`, con `WatermarkStrategy BoundedOutOfOrderness(5 min)` e `withIdleness(10s)`;
2. Filtro sulle sole compagnie target (filter);
3. Partizionamento dello stream tramite `keyBy(airline)`;
4. Applicazione di una finestra tumbling in event time di 1 ora (`TumblingEventTimeWindows.of(Time.hours(1))`);
5. Aggregazione tramite `ProcessWindowFunction` per ogni evento della finestra si incrementano i contatori di voli totali, completati, cancellati e devianti; per i soli voli non cancellati con `dep_delay` non nullo si accumulano somma e conteggio per la media, e si incrementa il conteggio delle partenze in ritardo (`dep_delay > 15`);
6. Calcolo delle metriche derivate alla chiusura della finestra (`dep_delay_mean`, `cancellation_rate`, `late_departure_rate`);
7. Scrittura dei risultati su due sink paralleli: file CSV (`Results/q1_output.csv`) con sink a parallelismo 1,

per garantire un solo writer e ordine di scrittura deterministico, e InfluxDB (per la dashboard Grafana), inoltre un altro sink dedicato alle metriche di latenza e throughput per finestra.

Q2 — Ranking degli Aeroporti per Ritardi Significativi

Q2 identifica, tra i voli completati, gli aeroporti di partenza con più ritardi significativi ($\text{dep_delay} > 30$ minuti), calcolando un ranking dei primi 10 su tre granularità temporali: sliding 1h, sliding 6h e una finestra globale cumulativa sull'intero dataset.

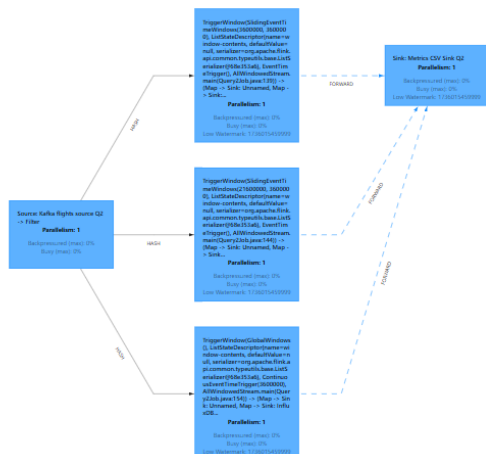


Figura 3: Schema Query 2

Il flusso di esecuzione:

1. Lettura dello stream da Kafka con lo stesso meccanismo di Q1, tramite un consumer group indipendente;
2. Filtro che scarta gli heartbeat, i voli cancellati o deviati e quelli con `origin_airport_id` nullo (`filter`);
3. Triplicazione della pipeline in tre rami paralleli, ciascuno con `windowAll` su una finestra diversa, l'intero flusso confluisce in un singolo operatore per finestra, che Flink esegue sempre a parallelismo 1 indipendentemente dal parallelismo configurato per il job:
 - 1) `SlidingEventTimeWindows(1h, slide 60min)` equivalente a una tumbling window;
 - 2) `SlidingEventTimeWindows(6h, slide 60min)`, scelta per produrre un ranking aggiornato ogni ora anche su una finestra di osservazione più ampia;
 - 3) `GlobalWindows` con `ContinuousEventTimeTrigger(60min)` che accumula tutti i voli dall'inizio dello stream e produce un ranking cumulativo aggiornato ogni ora di event time. Il trigger continua a scattare regolarmente anche durante le ore notturne grazie agli heartbeat.
4. Aggregazione per aeroporto all'interno di ciascuna `ProcessAllWindowFunction`, tramite una `HashMap<Integer, Q2AirportStats>` costruita a ogni trigger: per ogni evento si aggiornano conteggio voli, conteggio ritardi severi, somma e massimo del ritardo, e la lista dei voli più ritardati (solo i 20 peggiori);
5. Filtro di eleggibilità per la classifica (almeno 30 voli e almeno un ritardo severo nella finestra), ordinamento per numero di ritardi severi decrescente (a parità, per ID aeroporto crescente) e troncamento al top-10;
6. Scrittura dei risultati su CSV — con deduplicazione via `HashSet` per le finestre sliding, che possono produrre

righe duplicate causate dalle finestre sovrapposte, e senza deduplicazione per la finestra globale, le cui statistiche cumulative crescono ad ogni trigger — e su InfluxDB per Grafana.

La scelta di ricostruire la classifica da capo ad ogni trigger, anziché mantenere un accumulatore incrementale dedicato per la finestra globale, permette di riutilizzare senza modifiche la stessa struttura `Q2AirportStats` e la stessa logica di aggregazione già implementata per le due finestre sliding, a costo di un ricalcolo che cresce con la quantità di dati accumulati; questo compromesso è discusso in dettaglio in *Sezione 6* e ripreso tra gli sviluppi futuri in *Sezione 8*.

Da specificare, per la finestra globale, il timestamp riportato in output (`ts`) è fissato all'inizio semantico del dataset (2025-01-01T00:00:00Z), poiché la `GlobalWindow` non si chiude mai; inoltre, è stato registrato un timestamp separato, non esposto nel CSV richiesto ma usato solo per la serie temporale InfluxDB, che registra l'ora virtuale di ciascun trigger.

5. RISULTATI

Di seguito vengono presentati i grafici estratti dalla dashboard Grafana, che illustrano i risultati della query 1.

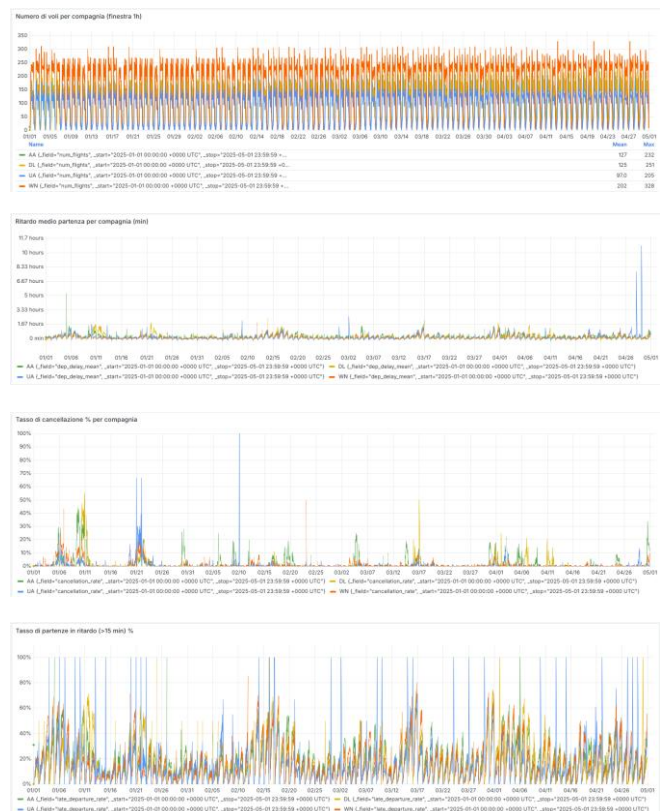
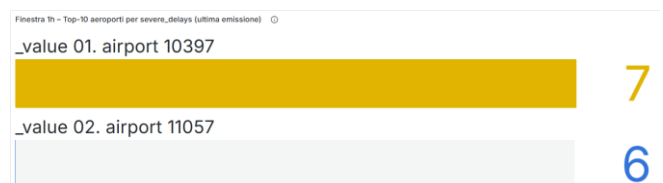


Figura 4: Risultati della Query 1

I grafici successivi, estratti dalla dashboard Grafana, mostrano i risultati della query 2.



7

6

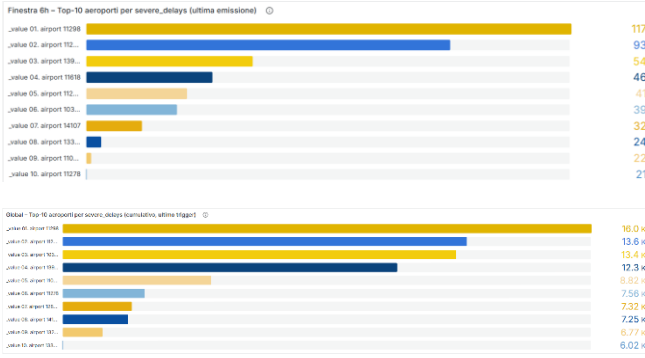


Figura 5: Risultati della Query 2

6. VALUTAZIONE SPERIMENTALE

A. Metodologia

Gli esperimenti sono stati condotti in **deploy locale** su un singolo host, con un processore Intel Core i5-1135G7 (11th Gen, 4 core fisici, 8 processori logici) e 16GB di RAM. Data la natura condivisa delle risorse tra tutti i container Docker (Kafka, Zookeeper, InfluxDB, Grafana, JobManager e TaskManager) su un ambiente WSL2 con risorse limitate, il parallelismo testato è stato volutamente contenuto fino a un massimo di 4: valori superiori avrebbero generato contesa tra i container per CPU e memoria, introducendo un overhead di sistema che avrebbe confuso l'effetto del parallelismo del job con quello della saturazione dell'host, rendendo i risultati meno interpretabili.

Sono stati condotti due esperimenti distinti sul parallelismo, ciascuno ripetuto 3 volte per configurazione con $\text{TIME_SCALE_FACTOR} = 86400$:

1. **Parallelismo del job** (flink run -p N, con $N=1,2,4$) su un singolo TaskManager multi-slot;
2. **Numero di TaskManager** (JVM separate, 1 slot ciascuna, $N=1,2,4$), per isolare l'effetto della distribuzione su processi distinti rispetto al semplice aumento di slot condivisi sulla stessa JVM.

Le metriche sono raccolte su **doppio canale**, per validarne reciprocamente l'affidabilità: un **canale esterno**, calcolato all'interno del job stesso (latenza come differenza tra il timestamp di emissione del risultato e il massimo kafka_produce_time tra gli eventi della finestra; throughput derivato dal numero di voli processati sulla durata effettiva della sessione, entrambi salvati su CSV), e un **canale interno**, ottenuto interrogando periodicamente le REST API di Flink per le metriche native dell'operatore di filtro (numRecordsOutPerSecond, EWMA) e per l'occupazione (busy time) dei task.

La **latenza** misurata dal canale esterno è quindi una latenza end-to-end in senso stretto: non riflette solo il tempo di elaborazione interno a Flink, ma include anche il transito su Kafka (dalla produzione al consumo del messaggio) e il tempo di buffering all'interno della finestra, cioè l'attesa dell'evento fino al trigger della finestra a cui appartiene. Il canale interno, basato sulle metriche native dell'operatore, non cattura invece questa componente a monte.

I due canali mostrano uno scarto relativo sistematico dell'ordine del 5–10%, spiegabile con la natura EWMA (pesata sui burst recenti) della metrica interna contro la media aritmetica sull'intera sessione del canale esterno.

Poiché il canale interno interroga le metriche dell'operatore **Source/Filtro** — comune e a monte della triplicazione di Q2 nei tre rami sliding 1h, sliding 6h e globale — il confronto tra i due canali avviene a livello di sistema: throughput e Busy% riportati per Q2 rappresentano quindi il carico complessivo del job, non quello specifico della sola finestra globale. Fa eccezione la latenza, per cui si riportano unicamente i risultati della finestra globale.

Il **Busy%** indica la percentuale di tempo in cui un operatore Flink è stato occupato a processare dati, rispetto al tempo totale disponibile. È la metrica più affidabile per isolare l'effetto del parallelismo, poiché throughput e latenza restano spesso entro il rumore di misura a fronte di variazioni di Busy% di un fattore 2 o più.

B. Soglia critica di parallelismo

I risultati discussi nelle sezioni successive possono essere inquadrati in un modello semplice a due regimi.

Sia λ il throughput sostenibile dal producer Python, una quantità dipendente dal TIME_SCALE_FACTOR adottato nella fase di replay, indipendente dal parallelismo configurato lato Flink.

Assumendo che il carico sia distribuito uniformemente sui p slot attivi, l'**utilizzo teorico di ciascuno slot** è dato da $\rho(p) = \lambda / (p \cdot \mu_1)$. Il parallelismo critico p_c , definito come il valore di p per cui $\rho(p) = 1$, vale quindi $p_c = \lambda / \mu_1$, dove μ_1 è la capacità di elaborazione di un singolo slot Flink.

Detto $\lambda_{target}(TSF)$ il **tasso di emissione teorico** richiesto per rispettare la compressione uniforme della scala temporale con fattore TSF, e detto μ_p il **tasso di emissione massima** effettivamente sostenibile dal producer Python (vincolata da single-thread/GIL, serializzazione e I/O verso Kafka), il **throughput** realmente osservato è:

$$\lambda(TSF) = \min(\lambda_{target}(TSF), \mu_p)$$

Poiché $\lambda_{target}(TSF)$ cresce linearmente con TSF, esiste un **valore critico** $TSF^* = \mu_p / \bar{\rho}$ (dove $\bar{\rho}$ è la densità media di eventi per secondo di event time nel dataset) oltre il quale il producer satura:

- per $TSF < TSF^*$, $\lambda(TSF) = \lambda_{target}(TSF)$ e il sistema è effettivamente vincolato dalla scelta del fattore di accelerazione;
- per $TSF \geq TSF^*$, $\lambda(TSF) = \mu_p$ e ulteriori incrementi di TSF non producono alcun aumento del throughput osservato, poiché il collo di bottiglia si è già spostato interamente sul producer.

Questa relazione ha una conseguenza diretta sulla **soglia critica di parallelismo del task slot** p_c : sostituendo $\lambda(TSF)$ nella definizione di p_c ,

$$p_c(TSF) = \lambda(TSF) / \mu_1$$

- Per $TSF < TSF^*$, p_c cresce linearmente con TSF: un'accelerazione maggiore richiede un parallelismo Flink proporzionalmente maggiore prima che il sistema transiti dal regime CPU-bound al regime producer-bound.

- Per $T_{SF} \geq T_{SF}^*$, invece, p_c si stabilizza al valore $p_c = \mu_p/\mu_1$, indipendente da ulteriori incrementi di T_{SF} .

Poiché $TIME_SCALE_FACTOR = 86400$ supera ampiamente la soglia critica T_{SF}^* , il sistema si trova, per l'intera valutazione sperimentale, in regime producer-bound — condizione confermata empiricamente dalla scalabilità piatta osservata nelle sezioni seguenti.

I valori riportati nelle tabelle seguenti sono la media aritmetica di 3 ripetizioni per configurazione. Per Q2, in base alla logica di implementazione, si è scelto di riportare i risultati della sola finestra globale: è l'unica delle tre granularità implementate il cui costo per trigger cresce con la quantità di dati accumulati e resta vincolata a parallelismo 1 indipendentemente dal parallelismo del job, il che la rende la componente più significativa ai fini dell'analisi di scalabilità; le finestre sliding 1h e 6h, più leggere, non mostrano invece variazioni rilevanti al variare del parallelismo.

C. Scalabilità a livello di Task Slot

Tabella 1: Scalabilità a livello di Task Slot

Query	Num. TS	Throughput (rps)	Latenza mean (ms)	Busy %
Q1	1	5099	167,4	9,17%
Q1	2	4874	169,5	4,10%
Q1	4	5029	171,7	4,23%
Q2 (global)	1	7304	1414	58,8%
Q2 (global)	2	7099	843	53,8%
Q2 (global)	4	6821	801	52,7%

Per **Q1**, il Busy% dimezza da $p=1$ a $p=2$ ma nell'intervallo testato non migliora ulteriormente a $p=4$. Questo è coerente con un sistema che, come confermato dal modello di *Sezione 6.B*, si trova già in regime producer-bound fin da $p=1$: il dimezzamento del Busy% riflette semplicemente la scala $1/p$ del carico distribuito su più slot, non l'attraversamento di una soglia — il collo di bottiglia resta a monte, nella capacità del producer Python di generare lo stream, per questo throughput e latenza restano piatti su tutta la scala testata. Un ulteriore parallelismo lato Flink non aiuta perché non c'è un vincolo di calcolo da alleggerire ulteriormente. A questo si aggiunge un limite strutturale della pipeline stessa: essendo partizionata per `keyBy(airline)` su sole 4 chiavi (AA, DL, UA, WN), un parallelismo superiore a 4 non avrebbe comunque lavoro aggiuntivo da distribuire — un vincolo teorico che non è stato verificato sperimentalmente, non avendo testato configurazioni oltre $p=4$.

Per **Q2**, il Busy% della finestra globale scende sensibilmente da $p=1$ a $p=2$ ma il guadagno oltre $p=2$ è marginale. La causa, qui, non è il numero di chiavi ma il limite architetturale introdotto in *Sezione 4*: `windowAll` sulla `GlobalWindows` viene sempre eseguita a parallelismo 1, indipendentemente dal parallelismo del job — il calo di Busy% da $p=1$ a $p=2$ riflette quindi la parallelizzazione degli operatori a monte (source, filtro), non della finestra globale stessa, che resta intrinsecamente seriale. La latenza mostra inoltre una distribuzione fortemente asimmetrica, effetto della natura $O(n)$ del ricalcolo ad ogni trigger: più tardi nel dataset, più costosa è ogni ricostruzione della classifica.

Di seguito, riportiamo i grafici di confronto tra il throughput ottenuto con il canale esterno e il canale interno, al variare del parallelismo, rispettivamente per Q1 e per la

finestra globale di Q2. In entrambi i casi lo scarto tra i due canali resta stabile nell'ordine del 5–10% indicato in *Sezione 6.A*, a conferma della coerenza reciproca delle due strumentazioni su tutta la scala di parallelismo testata.

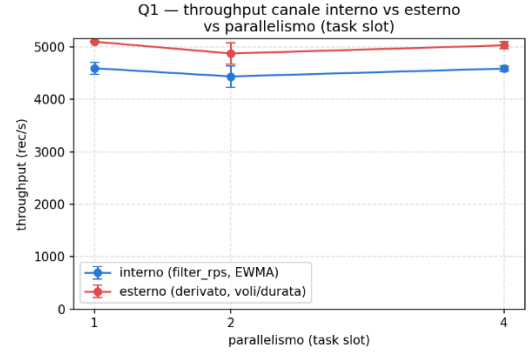


Figura 6: Throughput Q1 – scale TS – doppio canale

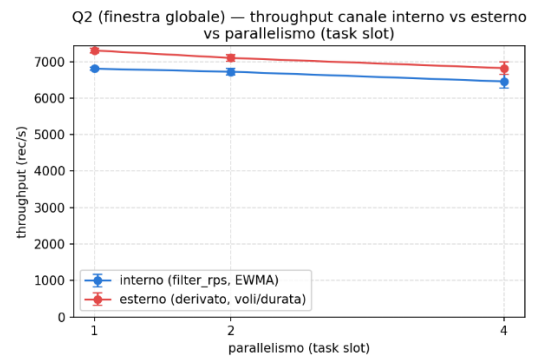


Figura 7: Throughput Q2 – scale TS – doppio canale

Applicando il modello introdotto nella *Sezione 6.B* ai dati di Tabella 1, e stimando μ_1 a partire dal Busy% osservato a $p=1$ ($\mu_1 \approx \lambda(1)/\rho(1)$, dove ρ indica l'utilizzo teorico di ciascun slot, assumendo che in regime producer-bound il throughput osservato approssimi λ), si ottiene per Q1 un parallelismo critico $p_c \approx 0,09$, e per Q2 (sull'operatore di filtro, comune ai tre rami) $p_c \approx 0,59$. Entrambi i valori, essendo inferiori a 1, indicano che il sistema si trova già nel **regime producer-bound** alla configurazione minima testata: per nessuna delle due query il parallelismo Flink risulta mai il fattore limitante nell'intervallo $p \in \{1, 2, 4\}$, il che spiega perché throughput e latenza restino sostanzialmente piatti su tutta la scala testata, mentre il Busy% ne segue fedelmente l'andamento decrescente atteso. Il valore di p_c più basso per Q1 rispetto a Q2 è coerente con la maggiore semplicità del filtro per compagnia rispetto al filtro di Q2, che deve gestire anche heartbeat, cancellazioni/deviazioni e valori nulli.

D. Scalabilità a livello di TaskManager

Tabella 2: Scalabilità a livello di TaskManager

Query	Num. TM	Throughput (rps)	Latenza mediana (ms)	Busy %
Q1	1	5093	168,7	9,27%
Q1	2	4893	171,0	4,14%
Q1	4	4753	172,0	4,07%
Q2 (global)	1	7130,16	233	61,5%
Q2 (global)	2	6985,94	132	49,2%
Q2 (global)	4	6535,76	132,5	49,6%

La Tabella 2 riporta gli stessi risultati ottenuti scalando questa volta il numero di TaskManager, ciascuno dotato di un solo task slot, anziché il parallelismo su una singola JVM multi-slot.

Il pattern osservato scalando sul numero di TaskManager conferma quanto emerso a livello di task slot. Il Busy% di Q1 dimezza da TM=1 a TM=2 (9,27% → 4,14%), mentre il calo residuo fino a TM=4 (4,07%) rientra nel rumore di misura (CV 1–2% in questo intervallo): un ulteriore conferma che il sistema è producer-bound sull'intera scala testata, non solo da TM=2 in poi. Lo stesso vincolo delle quattro chiavi di keyBy(airline), discusso in Sezione 6.C, resta valido anche in questo esperimento come limite teorico oltre la scala testata.

Per Q2 si osserva un andamento analogo: il Busy% della finestra globale scende nettamente da TM=1 a TM=2 (61,5% → 49,2%), per poi stabilizzarsi a TM=4 (49,6%), a conferma del vincolo di parallelismo 1 di windowAll. La latenza della finestra globale mostra però una distribuzione fortemente asimmetrica: a TM=1 la media supera i 2000 ms mentre la mediana resta a 233 ms, un effetto diretto della natura O(n) del ricalcolo ad ogni trigger — più tardi nel dataset, più costosa è ogni ricostruzione della classifica. Per questo motivo, coerentemente con quanto discusso in Sezione 6.A, in questa sezione si riporta la **mediana** come statistica di riferimento per la finestra globale.

Di seguito riportiamo i grafici che illustrano proprio questa divergenza, riportando media e mediana della latenza (canale esterno) al variare del numero di TaskManager, con barre d'errore a ± 1 deviazione standard sulle 3 ripetizioni.

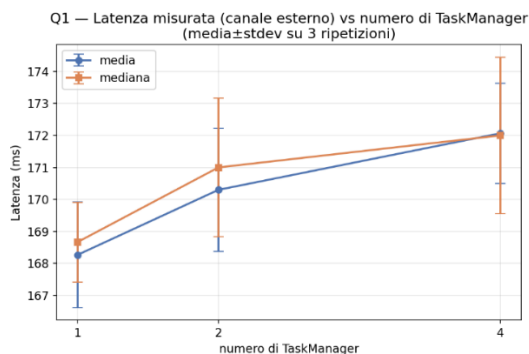


Figura 8: Latenza Q1 vs. numero di TM

Per Q1 (Fig. 8) le due statistiche restano molto vicine su tutto l'intervallo testato, e le barre d'errore si sovrappongono quasi completamente tra TM=2 e TM=4: la media resta quindi una statistica adeguata per questa query.

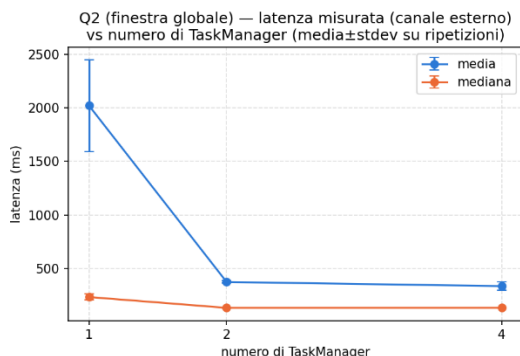


Figura 9: Latenza Q2 vs. numero di TM

Per Q2 (Fig. 9), invece, media e mediana divergono nettamente a TM=1 (2022,5 ms contro 233 ms), per poi riavvicinarsi bruscamente da TM=1 a TM=2 (373,7 ms contro 132 ms) e restare sostanzialmente piatte a TM=4 (335,2 ms e 132,5 ms). Questa visualizzazione chiarisce il motivo della scelta della mediana come metrica di riferimento per la finestra globale: a bassi livelli di parallelismo la media viene distorta da pochi trigger tardivi molto più costosi degli altri, restituendo un'immagine poco rappresentativa delle prestazioni reali del sistema.

Il throughput mostra invece un comportamento diverso rispetto allo scaling su task slot: cala in modo consistente all'aumentare del numero di TaskManager per entrambe le query, da 5093 a 4753 rps per Q1 (circa 7%), e da 7130 a 6536 rps per Q2 (circa 8%). La differenza è coerente con la natura stessa delle due modalità di scaling: distribuire il lavoro su JVM distinte introduce comunicazione di rete reale tra processi per lo shuffle dei dati, oltre all'overhead di gestione di ciascuna JVM aggiuntiva (heap, garbage collection, thread di sistema); su un host con risorse condivise tra tutti i container Docker, questo costo si somma alla contesa già esistente per CPU e memoria e diventa più visibile al crescere del numero di TaskManager. La coerenza dell'effetto su entrambe le query rafforza questa lettura, pur restando un'ipotesi non isolabile con certezza dai soli dati raccolti; è comunque coerente con la scelta di limitare gli esperimenti a un massimo di 4 TaskManager.

Di seguito, riportiamo i grafici di confronto tra canale interno ed esterno del throughput al variare del numero di TaskManager, rispettivamente per Q1 e per la finestra globale di Q2. Anche in questo caso lo scarto tra due canale si mantiene nell'ordine del 5–10%, confermando la coerenza delle due misurazioni indipendentemente dalla modalità di scaling adottata.

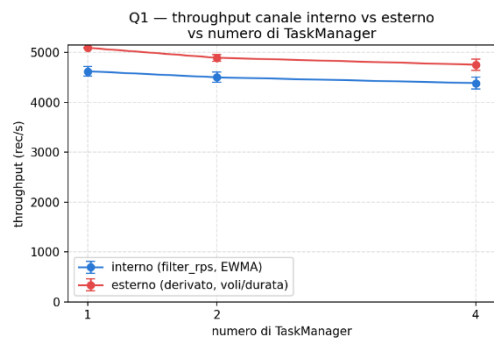


Figura 10: Throughput Q1 – scale TM – doppio canale

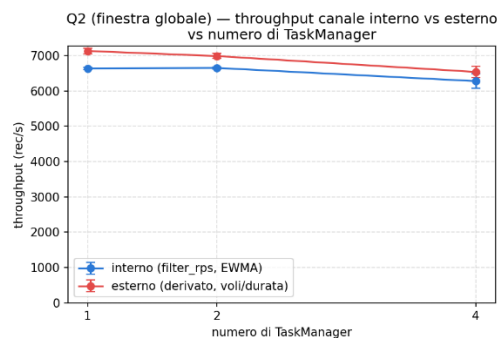


Figura 11: Throughput Q2 – scale TM – doppio canale

Il lieve calo di throughput osservato in entrambi i grafici al crescere del numero di TaskManager è coerente con l'overhead di coordinazione discusso sopra.

7. CONCLUSIONI

Il progetto ha dimostrato la fattibilità di un sistema di stream processing end-to-end per l'analisi di dati sui voli, dall'ingestion via Kafka fino alla visualizzazione su Grafana, gestendo correttamente le complessità tipiche dell'elaborazione in event time — disordine degli eventi, gap temporali privi di traffico, watermarking distribuito.

L'aspetto più significativo emerso dalla valutazione sperimentale non è tanto la misura assoluta delle prestazioni, quanto la loro spiegazione: in entrambe le query il parallelismo Flink non è mai risultato il fattore limitante, poiché il sistema si è rivelato producer-bound sull'intera scala testata ($p_c < 1$ per entrambe le query). A questo si affiancano vincoli di design specifici di ciascuna pipeline — la granularità del partizionamento per Q1 (4 chiavi), la natura intrinsecamente seriale dell'operatore windowAll per Q2 — che pongono un limite teorico ulteriore alla scalabilità utile, non ancora osservabile nell'intervallo di parallelismo testato. Questo suggerisce che, in sistemi di questo tipo, la scalabilità va progettata a monte insieme alla logica applicativa e al dimensionamento dell'intera pipeline (producer incluso), e non trattata come una proprietà che si ottiene semplicemente aumentando le risorse a valle.

Il principale limite del lavoro resta la scala dell'ambiente di test: un singolo host con risorse condivise tra tutti i container non consente di distinguere con certezza gli effetti dovuti al parallelismo di Flink da quelli dovuti alla contesa di sistema, né di verificare sperimentalmente i vincoli architetturali oltre $p=4$.

8. SVILUPPI FUTURI

Le direzioni di lavoro più naturali sono i seguenti:

- La prima è la validazione su un ambiente distribuito reale (cluster multi-nodo), che permetterebbe di

isolare gli effetti architetturali osservati dalla contesa di risorse locale e di testare configurazioni di parallelismo oltre quelle qui esplorate.

- La seconda riguarda l'estensione funzionale del sistema: l'implementazione di Q3 e l'aggiunta di checkpointing e fault tolerance permetterebbero di valutarne la robustezza in presenza di guasti, aspetto non coperto da questo sistema implementato.
- La terza, più immediata, è la rimozione del collo di bottiglia lato producer — tramite TSF o partizionamento del topic Kafka — condizione necessaria per osservare pienamente, sulle metriche di sistema, l'effetto della scalabilità di Flink al di là del plateau riscontrato a $p=2$.
- La quarta, più mirata alla componente più critica emersa nella valutazione sperimentale, è l'implementazione di un accumulatore incrementale (AggregateFunction) per la finestra globale di Q2, in sostituzione dell'attuale ricostruzione completa della classifica ad ogni trigger (*Sezione 4*): eliminerebbe la crescita $O(n)$ del costo di elaborazione documentata in *Sezione 6*, al prezzo di duplicare parzialmente la logica di aggregazione attualmente condivisa con le finestre sliding.

REFERENCES

- [1] Apache Software Foundation. Apache Flink. Available: <https://flink.apache.org/>
- [2] U.S. Department of Transportation. Bureau of Transportation Statistics (BTS). Available: <https://www.transtats.bts.gov/>
- [3] Apache Software Foundation. Apache Kafka. Available: <https://kafka.apache.org/>
- [4] Docker. Available: <https://www.docker.com/>
- [5] InfluxDB. Available: <https://www.influxdata.com/>
- [6] Grafana Labs. Grafana: The open observability platform. Available: <https://grafana.com/grafana/>